

Tài liệu Giải thích Module Dashboard, Notification và Badges

FitLife — Người 4

Ngày 7 tháng 7 năm 2026

Mục lục

0.1	Module này làm gì?	2
0.2	Kiến trúc tổng quan (dễ hiểu)	2
0.3	Dashboard (Trang chủ tổng quan)	3
0.3.1	User thấy gì trên Dashboard?	3
0.3.2	Dữ liệu lấy từ đâu?	3
0.3.3	Biểu đồ 7 ngày hoạt động thể nào?	4
0.3.4	Nếu RPC lỗi thì sao?	5
0.3.5	File code liên quan	5
0.4	Badges (Huy hiệu thành tích)	5
0.4.1	Huy hiệu là gì?	5
0.4.2	Danh sách 8 huy hiệu mặc định	5
0.4.3	Cơ chế cấp huy hiệu (từng bước)	6
0.4.4	Cấu trúc database	6
0.4.5	File code liên quan	6
0.5	Notification (Thông báo & nhắc nhở)	7
0.5.1	Ba loại thông báo	7
0.5.2	User bật/tắt ở đâu?	7
0.5.3	Luồng bật nhắc tập (ví dụ)	8
0.5.4	Lưu ý quan trọng về Expo Go	8
0.5.5	Khôi phục nhắc khi mở lại app	8
0.5.6	File code liên quan	9
0.6	Luồng hoạt động tổng hợp	9
0.6.1	Kịch bản A: User mở app buổi sáng	9
0.6.2	Kịch bản B: User hoàn thành buổi tập	9

0.6.3	Kịch bản C: User bật nhắc uống nước	10
0.7	Backend (Database)	10
0.7.1	Migration cần chạy	10
0.7.2	Bảng và cột module tạo thêm	10
0.7.3	Bảo mật	10
0.8	Kiểm thử	11
0.8.1	Lệnh chạy test tự động	11
0.8.2	Checklist test thủ công	11
0.9	Tóm tắt nhanh	11
0.10	Giải thích code & hàm chi tiết	12
0.10.1	getDashboardSummary(userId) — dashboardService.ts . .	12
0.10.2	normalizeWeeklyWorkouts(raw) — dashboardService.ts . .	13
0.10.3	buildDashboardSummaryClient(userId) — fallback client . .	14
0.10.4	loadDashboard() — DashboardScreen.tsx	14
0.10.5	WeeklyProgressChart — biểu đồ 7 ngày	15
0.10.6	isBadgeEarned(badge, stats) — badgeCalculations.ts . .	16
0.10.7	loadBadgeStats(userId) — badgeService.ts	17
0.10.8	syncUserBadges(userId) — cấp huy hiệu tự động	17
0.10.9	getNotifications() — lazy-load an toàn Expo Go	18
0.10.10	updateNotificationPreferences(userId, prefs) — bật/tắt nhắc	19
0.10.11	syncAllRemindersOnLaunch(userId) — khôi phục khi mở app	20
0.10.12	RPC get_dashboard_summary() — SQL trên server	20
0.10.13	Tóm tắt hàm	21

Dự án: FitLife — Ứng dụng tập luyện & sức khỏe
Phạm vi: Module của Người 4 (theo bản phân công nhóm)
Công nghệ: React Native (Expo) + Supabase (PostgreSQL)

0.1 Module này làm gì?

Module **Dashboard, Notification & Badges** là phần giúp người dùng **nhìn tổng quan tiến độ, nhận nhắc nhở và được ghi nhận thành tích** khi tập luyện đều đặn.

Gồm **3 chức năng chính**:

Chức năng	Người dùng thấy gì	Mục đích
Dashboard	Tab Home: chuỗi tập, calo, số buổi tập, biểu đồ 7 ngày, sức khỏe và huy hiệu	Tổng hợp số liệu quan trọng trên một màn hình
Notification	Thông báo nhắc uống nước, nhắc tập, thông báo huy hiệu mới	Giúp user không quên tập và uống nước
Badges	Lưới huy hiệu (Khởi đầu, Kiên trì 3 ngày, Đốt cháy 500 kcal...)	Gamification — khuyến khích duy trì thói quen

Người 4 phụ trách cả 3 lớp: - **Frontend** — giao diện Dashboard, huy hiệu, cài đặt nhắc nhở - **Backend** — API tổng hợp Dashboard, bảng huy hiệu, lưu cài đặt thông báo - **Test** — kiểm thử thông báo, quyền OS, logic huy hiệu

0.2 Kiến trúc tổng quan (dễ hiểu)

Hãy tưởng tượng app như một nhà 3 tầng:

```
+-----+
| TANG 1 - GIAO DIEN (man hinh user nhìn) |
| DashboardScreen, BadgeGrid, Modal nhắc |
+-----+
|                                           |
|         | gọi hàm                         |
|         +-----v-----+
| TANG 2 - SERVICE (xu ly nghiệp vụ)      |
| dashboardService, badgeService,         |
| notificationService, waterReminder      |
+-----+
```

	gọi API / RPC	
+	-----v-----+	
	TANG 3 - SUPABASE (lưu trữ & bảo mật)	
	PostgreSQL + RLS + RPC functions	
+	-----+	

Quy tắc quan trọng: - Màn hình **không** gọi database trực tiếp — luôn qua **service**. - Logic tính toán thuần (vd: đủ điều kiện huy hiệu chưa?) nằm trong `lib/` để dễ test. - Mỗi user **chỉ thấy dữ liệu của mình** nhờ RLS (Row Level Security) trên Supabase.

0.3 Dashboard (Trang chủ tổng quan)

0.3.1 User thấy gì trên Dashboard?

Khi mở tab **Home**, màn hình hiển thị:

1. **Lưới chỉ số nhanh** - streak hiện tại, calo đốt hôm nay, số buổi tập hôm nay và streak dài nhất.
2. **Biểu đồ 7 ngày** — Hoạt động tập luyện trong tuần qua (mỗi cột = 1 ngày)
3. **Thẻ buổi tập** - hiện đang là dữ liệu trình diễn cố định (Power Strength II, 12/24, 50%), chưa đọc chương trình hiện tại từ database.
4. **Sức khỏe hôm nay** - giờ nhắc tập, calo nạp, nước và macro dinh dưỡng.
5. **Huy hiệu** - các badge đã đạt (sáng) và chưa đạt (mờ).

Kéo xuống để **làm mới** (pull-to-refresh). Sau khi hoàn thành buổi tập, Dashboard **tự cập nhật** số liệu mới.

0.3.2 Dữ liệu lấy từ đâu?

Riêng phần **số liệu tổng quan**, app ưu tiên một RPC trên server thay vì đọc từng bảng: `get_dashboard_summary()` — hàm PostgreSQL trả về JSON gồm:

Trường	Ý nghĩa
<code>display_name</code>	Tên hiển thị
<code>current_streak</code>	Chuỗi ngày tập liên tiếp hiện tại
<code>longest_streak</code>	Kỷ lục chuỗi dài nhất
<code>calories_burned_today</code>	Calo đốt hôm nay
<code>workouts_today</code>	Số buổi tập hôm nay

Trường	Ý nghĩa
weekly_workouts	Mảng 7 ngày: mỗi ngày có date, workouts, calories_burned
wakeup_time	Giờ dậy (dùng cho nhắc tập)
Các cờ *_reminder_enabled	Đã bật nhắc nước/tập/badge chưa

Ví dụ JSON trả về:

```
{
  "display_name": "Minh Sang",
  "current_streak": 3,
  "longest_streak": 5,
  "calories_burned_today": 320,
  "workouts_today": 1,
  "weekly_workouts": [
    { "date": "2026-06-24", "workouts": 0, "calories_burned": 0 },
    { "date": "2026-06-25", "workouts": 1, "calories_burned": 280 },
    { "date": "2026-06-26", "workouts": 0, "calories_burned": 0 },
    { "date": "2026-06-27", "workouts": 1, "calories_burned": 310 },
    { "date": "2026-06-28", "workouts": 1, "calories_burned": 290 },
    { "date": "2026-06-29", "workouts": 0, "calories_burned": 0 },
    { "date": "2026-06-30", "workouts": 1, "calories_burned": 320 }
  ],
  "wakeup_time": "06:30",
  "water_reminder_enabled": true,
  "workout_reminder_enabled": false,
  "badge_notifications_enabled": true
}
```

Dashboard vẫn gọi thêm luồng huy hiệu và cài đặt thông báo song song; vì vậy không nên hiểu là toàn bộ màn hình chỉ có đúng một request.

0.3.3 Biểu đồ 7 ngày hoạt động thế nào?

Vấn đề đã gặp: Khi user chưa tập buổi nào, biểu đồ hiển thị cột rất mờ — trông như lỗi.

Cách xử lý: - Server luôn trả **đủ 7 ngày** (kể cả ngày không tập = 0 buổi, 0 calo). - Client chuẩn hóa lại qua hàm `normalizeWeeklyWorkouts()` — khớp đúng ngày, điền 0 nếu thiếu. - Khi chưa có buổi tập nào -> hiển thị thông báo **“Chưa có buổi tập nào”** thay vì cột trống mờ. - Ngày hôm nay được **highlight** màu xanh lá; nhãn CN-T7 luôn hiển thị.

Lưu ý múi giờ: RPC dùng `current_date` theo timezone của database, còn client tạo khung 7 ngày theo timezone thiết bị. Nếu hai timezone khác nhau, dữ liệu quanh nửa đêm

có thể lệch một ngày. Cần thống nhất timezone hoặc truyền ngày local vào RPC nếu muốn xử lý triệt để.

0.3.4 Nếu RPC lỗi thì sao?

App thử **phương án dự phòng (fallback)**: dashboardService đọc profiles, user_streaks và workout_sessions rồi gom số liệu giống RPC. Fallback hữu ích khi RPC chưa tồn tại hoặc trả lỗi, nhưng không bảo đảm cứu được trường hợp mất mạng vì các truy vấn dự phòng cũng cần Supabase.

0.3.5 File code liên quan

File	Vai trò
src/screens/DashboardScreen.tsx	Màn hình chính, gọi load dữ liệu
src/services/dashboardService.ts	Gọi RPC + chuẩn hóa 7 ngày
src/components/StatsGrid.tsx	Thẻ sức khỏe: giờ nhắc, dinh dưỡng, nước và macro
src/components/ WeeklyProgressChart.tsx	Biểu đồ cột 7 ngày
src/components/WorkoutCard.tsx	Thẻ buổi tập trình diễn; nội dung hiện đang hard-code

0.4 Badges (Huy hiệu thành tích)

0.4.1 Huy hiệu là gì?

Giống huy chương trong game: khi user đạt mốc nhất định, app **tự cấp huy hiệu** và hiển thị sáng trên Dashboard.

0.4.2 Danh sách 8 huy hiệu mặc định

Tên	Điều kiện đạt
Khởi đầu	Hoàn thành onboarding lần đầu
Buổi tập đầu	Hoàn thành 1 buổi tập
Kiên trì 3 ngày	Tập liên tiếp 3 ngày (streak ≥ 3)
Tuần vàng	Tập liên tiếp 7 ngày

Tên	Điều kiện đạt
Thép không gỉ	Tập liên tiếp 30 ngày
Hydration Pro	Đạt mục tiêu uống nước trong 1 ngày
Dinh dưỡng chuẩn	Đạt mục tiêu calo ăn trong 1 ngày
Đốt cháy	Đốt ≥ 500 kcal trong một ngày

0.4.3 Cơ chế cấp huy hiệu (từng bước)

```
Moi lan mo Dashboard
|
v
syncUserBadges(userId)
|
|-> Thu thập thông tin user:
|   - Đã onboarding chưa?
|   - Tổng số buổi tập?
|   - Streak hiện tại?
|   - Bao nhiêu ngày đạt mục tiêu nước/calor?
|   - Ngày đốt calo nhiều nhất?
|
|-> So sánh với từng huy hiệu (hàm isBadgeEarned)
|
|-> Huy hiệu đủ điều kiện mà chưa có -> INSERT vào user_badges
|
|-> Tra danh sách badge (đã đạt / chưa đạt) cho UI
```

Lưu ý: Unique constraint (user_id, badge_id) bảo đảm mỗi huy hiệu chỉ được lưu một lần. Service bỏ qua riêng lỗi 23505 khi hai lần đồng bộ cùng insert một badge; các lỗi insert khác được ghi cảnh báo.

0.4.4 Cấu trúc database

Bảng badges — danh mục huy hiệu (dùng chung cho mọi user): - code — mã cố định (vd: streak_7) - title, description — tên và mô tả hiển thị - criteria_type — loại điều kiện (streak, workout_sessions, ...) - criteria_value — ngưỡng (vd: 7 ngày)

Bảng user_badges — huy hiệu user đã đạt: - user_id + badge_id — mỗi cặp chỉ có 1 lần - earned_at — thời điểm đạt

0.4.5 File code liên quan

File	Vai trò
src/services/badgeService.ts	Đồng bộ và cấp huy hiệu
src/lib/badgeCalculations.ts	Logic kiểm tra đủ điều kiện
src/lib/badgeCatalog.ts	Danh mục fallback 4 badge khi không tải được bảng badges
src/components/BadgeGrid.tsx	Hiển thị lưới huy hiệu
src/lib/__tests__/badgeLogic.test.ts	Unit test logic huy hiệu

Danh mục chính trong migration có **8 badge**, nhưng `FALLBACK_BADGES` hiện chỉ có **4 badge đầu**. Fallback này cũng vẫn cần các truy vấn Supabase để tính thống kê, nên không phải chế độ offline hoàn chỉnh.

0.5 Notification (Thông báo & nhắc nhở)

0.5.1 Ba loại thông báo

Loại	Khi nào bắn	Lịch
Nhắc uống nước	User bật trong cài đặt	8 mốc/ngày: 7h, 9h, 11h, 13h, 15h, 17h, 19h, 21h
Nhắc tập luyện	User bật trong cài đặt	Đúng giờ <code>wakeup_time</code> user chọn (vd 6:30 sáng)
Huy hiệu mới	Dashboard phát hiện badge đạt mà state trước đó chưa có	Local notification ngay lập tức

0.5.2 User bật/tắt ở đâu?

User có thể mở `NotificationSettingsModal` từ menu **3 gạch** > **Thông báo** hoặc nút chuông trên Dashboard:

- Công tắc nhắc uống nước
- Công tắc nhắc tập luyện (+ chọn giờ)

- Công tắc thông báo huy hiệu

Ba cờ bật/tắt và `wakeup_time` được lưu trong `profiles`. Cờ có thể đồng bộ giữa thiết bị, nhưng quyền OS và id lịch thông báo nằm cục bộ trên từng máy trong `AsyncStorage`.

0.5.3 Luồng bật nhắc tập (ví dụ)

```
User bat "Nhac tap luyen"
|
v
App xin quyền thông báo tu hệ điều hành (iOS/Android)
|
+-- User tu chôi -> báo lỗi, không bat
|
\-- User đồng ý
    |
    v
    Lap lịch thông báo lập hàng ngày dùng gio wakeup_time
    |
    v
    Lưu id thông báo vào AsyncStorage (để huy sau này)
    |
    v
    Ghi workout_reminder_enabled = true vào profiles
```

Mỗi ngày đúng giờ, điện thoại hiển thị: **“Giờ tập luyện — Đã đến giờ tập!”**

0.5.4 Lưu ý quan trọng về Expo Go

Thông báo KHÔNG hoạt động trên Expo Go (app test qua QR code). Chỉ chạy trên: - **Development build** (build riêng qua EAS) - **Bản release** (APK/IPA cài trên máy)

Code xử lý an toàn: trên Expo Go, module thông báo **không load** — app không crash, chỉ hiển thị thông báo hướng dẫn khi user cố bật nhắc.

0.5.5 Khôi phục nhắc khi mở lại app

Sau khi đăng nhập và hoàn thành onboarding, `App.tsx` gọi `syncAllRemindersOnLaunch()`: - Đọc cờ từ `profiles` (user đã bật nhắc nước/tập chưa) - Nếu đã bật -> đặt lại lịch thông báo trên thiết bị

Sau khi cài lại hoặc xóa dữ liệu, app sẽ **thử** tạo lại lịch từ cờ server. Việc khôi phục còn phụ thuộc quyền thông báo trên thiết bị và kết nối Supabase.

Giới hạn hiện tại của thông báo badge: `notifyBadgeEarned()` không tự xin quyền OS. Ngoài ra, state badge ban đầu là mảng rỗng nên lần mount Dashboard đầu tiên có thể coi

mọi badge đã đạt là “mới” và bắn lại thông báo. Muốn đúng nghĩa “chỉ một lần”, cần so sánh với tập badge trước khi gọi `syncUserBadges` hoặc dùng danh sách id vừa insert.

0.5.6 File code liên quan

File	Vai trò
<code>src/services/notificationService.ts</code>	Nhắc tập, prefs, thông báo badge
<code>src/services/waterReminderService.ts</code>	Nhắc uống nước (8 mốc)
<code>src/components/NotificationSettingsModal.tsx</code>	Giao diện bật/tắt
<code>src/components/AppMenuDrawer.tsx</code>	Menu mở modal nhắc nhở

0.6 Luồng hoạt động tổng hợp

0.6.1 Kịch bản A: User mở app buổi sáng

1. Đăng nhập -> đã onboarding
2. App đóng bỏ nhắc nhở (nếu đã bật trước đó)
3. Vào tab Home (Dashboard)
4. Tại song song:
 - So liệu Dashboard (RPC)
 - Đóng bỏ huy hiệu (cập mới nếu đủ điều kiện)
 - Đọc cài đặt thông báo
5. Hiện thị: calo, biểu đồ 7 ngày, huy hiệu
6. Nếu state trước chưa có badge đạt -> thu phát local notification "Bạn vừa mở khóa ..."

0.6.2 Kịch bản B: User hoàn thành buổi tập

1. Làm xong bài tập cuối trong `WorkoutDetailScreen`
2. Server ghi `workout_session` + cập nhật streak
3. App tăng `refreshKey` -> Dashboard tải lại
4. Biểu đồ thêm cột hôm nay, calo tăng
5. `syncUserBadges` có thể cập "Buổi tập đầu", "Kiến trì 3 ngày"...
6. Thu phát thông báo huy hiệu (nếu bật; hiện có rủi ro ban lại khi mount)

0.6.3 Kịch bản C: User bật nhắc uống nước

1. Menu 3 gạch -> Thông báo -> Bật "Nhắc uống nước"
2. Xin quyền OS -> lập 8 thông báo lập ngay
3. Lưu có `water_reminder_enabled = true`
4. Từ 7h-21h, mỗi 2 tiếng nhận: "Đã đến giờ uống nước!"

0.7 Backend (Database)

0.7.1 Migration cần chạy

File: `supabase/migrations/20260627200000_dashboard_badges_notifications.sql`

Phải chạy SAU migration workout (tạo bảng `profiles`, `workout_sessions`, `user_streaks`).

Thứ tự đầy đủ:

Bước	File migration
1	<code>20260627000000_nutrition_water_health.sql</code>
2	<code>20260627173357_workout_onboarding_atomic_updates.sql</code>
3	<code>20260627200000_dashboard_badges_notifications.sql</code> <- module này
4	<code>20260701100000_add_new_programs_v2.sql</code>

0.7.2 Bảng và cột module tạo thêm

Mở rộng profiles: - `workout_reminder_enabled` — bật nhắc tập (mặc định: tắt) - `badge_notifications_enabled` — thông báo huy hiệu (mặc định: bật)

Bảng mới: - `badges` — 8 huy hiệu mặc định - `user_badges` — huy hiệu user đã đạt

Hàm RPC mới: - `get_dashboard_summary()` — trả JSON tổng hợp Dashboard

0.7.3 Bảo mật

- **RLS:** User chỉ xem/cấp huy hiệu của chính mình; danh mục `badges` chỉ role `authenticated` được đọc.
- **RPC:** Chạy với quyền người gọi (`SECURITY INVOKER`), từ chối nếu chưa đăng nhập.
- Mỗi user **không thể** xem streak/calor/huy hiệu của user khác.

- Migration đã có GRANT tường minh cho Data API, phù hợp với thay đổi mặc định của Supabase năm 2026.

0.8 Kiểm thử

0.8.1 Lệnh chạy test tự động

```
npm run test:badges      # test logic huy hieu
npm run test:weekly      # test normalizeWeeklyWorkouts
npm test                 # chạy tất cả test
npm run typecheck        # kiểm tra TypeScript
```

0.8.2 Checklist test thủ công

Dashboard: - ☐ User mới (0 buổi tập): biểu đồ hiển thị “Chưa có buổi tập”, không cột mờ - ☐ Có buổi tập: cột đúng ngày, hôm nay highlight xanh - ☐ Kéo xuống refresh -> số liệu cập nhật - ☐ Sau khi tập xong -> Dashboard tự refresh

Huy hiệu: - ☐ Tập buổi đầu -> nhận “Buổi tập đầu” - ☐ Streak 3 ngày -> nhận “Kiên trì 3 ngày” - ☐ Huy hiệu đã đạt sáng, chưa đạt mờ

Thông báo: - ☐ Expo Go: bật nhắc -> báo lỗi hướng dẫn, không crash - ☐ Dev build: bật nhắc nước -> nhận đúng 8 mốc - ☐ Dev build: bật nhắc tập -> nhận đúng giờ đã chọn - ☐ Đạt huy hiệu mới + bật thông báo badge -> có thông báo đẩy - ☐ Mount lại Dashboard không bắn lại các badge đã đạt từ trước - ☐ Từ chối quyền OS -> cờ server và lịch local không rơi vào trạng thái lệch nhau - ☐ Kiểm tra dữ liệu 7 ngày quanh 00:00 khi timezone database khác timezone thiết bị

Phạm vi test tự động hiện tại: badgeLogic.test.ts mới kiểm tra 2 trường hợp của badge streak (đạt/chưa đạt mốc 3). Chưa có test tự động cho 5 loại điều kiện còn lại, RPC/fallback, RLS, lịch thông báo và tình huống đồng bộ cạnh tranh.

0.9 Tóm tắt nhanh

Câu hỏi	Trả lời ngắn
Dashboard lấy data từ đâu?	Số liệu chính từ RPC <code>get_dashboard_summary()</code> ; badge và prefs là các request riêng

Câu hỏi	Trả lời ngắn
Huy hiệu cập khi nào?	Mỗi lần mở Dashboard, <code>syncUserBadges</code> kiểm tra và cập tự động
Nhắc nhở lưu ở đâu?	Cờ/giờ trong <code>profiles</code> ; id lịch và quyền OS ở từng thiết bị
Tại sao Expo Go không có thông báo?	Giới hạn của Expo Go — cần development build
Biểu đồ 7 ngày trống?	Đã fix: empty state + luôn đủ 7 ngày từ server
File migration module?	<code>20260627200000_dashboard_badges_notifications.sql</code>

0.10 Giải thích code & hàm chi tiết

Phần này giải thích **từng hàm quan trọng**: đầu vào, đầu ra, từng bước xử lý và lý do thiết kế.

0.10.1 `getDashboardSummary(userId)` — `dashboardService.ts`

Mục đích: Lấy toàn bộ số liệu Dashboard trong một lần gọi.

RPC tự lấy user từ JWT bằng `auth.uid()`; tham số `userId` chỉ được dùng khi chuyển sang fallback client.

Code:

```
export async function getDashboardSummary(userId: string): Promise<
  DashboardSummary> {
  const { data, error } = await supabase.rpc('get_dashboard_summary');
  if (!error && data) {
    const summary = data as DashboardSummary;
    return {
      ...summary,
      weekly_workouts: normalizeWeeklyWorkouts(summary.weekly_workouts),
    };
  }
  return buildDashboardSummaryClient(userId);
}
```

Cách hoạt động (từng bước):

1. `supabase.rpc('get_dashboard_summary')` gọi hàm PostgreSQL; Supabase client gửi JWT của session hiện tại.

2. Khi có data và không có error, app dùng kết quả RPC.
3. `normalizeWeeklyWorkouts(...)` chuẩn hóa mảng 7 ngày trước khi render.
4. Khi RPC lỗi hoặc không có data, `buildDashboardSummaryClient(userId)` thử đọc từng bảng và gom số liệu.

Tại sao có fallback? User vẫn có thể thấy Dashboard khi migration RPC chưa chạy. Nếu nguyên nhân là mất mạng hoặc lỗi quyền truy cập chung, fallback cũng có thể thất bại.

0.10.2 `normalizeWeeklyWorkouts(raw)` — `dashboardService.ts`

Mục đích: Đảm bảo biểu đồ **luôn nhận đúng 7 ngày**, kể cả khi server trả thiếu hoặc date kèm timestamp.

Code:

```
export function normalizeWeeklyWorkouts(raw: unknown): WeeklyWorkoutDay[] {
  const base = buildEmptyWeekly(); // tao san 7 ngay, moi ngay workouts=0,
    calories=0
  if (!Array.isArray(raw) || raw.length === 0) return base;

  const map = new Map(base.map((day) => [day.date, { ...day }]));
  for (const item of raw) {
    if (!item || typeof item !== 'object') continue;
    const row = item as Record<string, unknown>;
    const date = normalizeDateString(String(row.date ?? '')); // "2026-06-30
    00:00:00" -> "2026-06-30"
    if (!map.has(date)) continue;
    map.set(date, {
      date,
      workouts: Number(row.workouts ?? 0),
      calories_burned: Number(row.calories_burned ?? 0),
    });
  }
  return base.map((day) => map.get(day.date) ?? day);
}
```

Ví dụ minh họa:

```
Input tu server (thieu ngay, date lech format):
[ { date: "2026-06-30 00:00:00", workouts: 1, calories_burned: 320 } ]

buildEmptyWeekly() tao base 7 ngay: 24/6 -> 30/6, tat ca = 0
```

Sau khi merge vào map:

30/6 -> workouts: 1, calories: 320
cac ngay khac -> giu 0

Output: mang 7 phan tu, dung thu tu ngay

Hàm phụ trợ `buildEmptyWeekly()`: Dùng `localDateDaysAgo(6 - index)` — tính ngày theo **timezone máy**, không dùng UTC (tránh lệch ngày khi user ở VN).

0.10.3 `buildDashboardSummaryClient(userId)` — **fallback client**

Mục đích: Tự gom số liệu khi RPC không dùng được.

Cách hoạt động:

`Promise.all` (chạy song song 3 query):

```
+-- getProfile(userId)           -> display_name, wakeup_time, co nhac nho
+-- user_streaks                 -> current_streak, longest_streak
\-- workout_sessions (7 ngay)   -> workout_date, total_calories
```

Duyệt tung session -> cong don vao weeklyMap theo ngay

Loc session hom nay -> tinh calories_burned_today, workouts_today

Tra object DashboardSummary giống het RPC

Ưu điểm `Promise.all`: 3 query chạy **đồng thời** thay vì tuần tự; tổng thời gian thường gần với query chậm nhất, nhưng không có bảo đảm cố định là nhanh hơn 3 lần.

0.10.4 `loadDashboard()` — **DashboardScreen.tsx**

Mục đích: Hàm trung tâm tải và hiển thị toàn bộ Dashboard.

Code:

```
const loadDashboard = useCallback(async () => {
  const [dashboard, badgeList, prefs] = await Promise.all([
    getDashboardSummary(userId),
    syncUserBadges(userId),
    getNotificationPreferences(userId).catch(() => null),
  ]);

  setSummary(dashboard);
  setBadges((previous) => {
    if (prefs?.badge_notifications_enabled) {
```

```

    const previousEarned = new Set(previous.filter(b => b.earned).map(b => b.id)
    );
    for (const badge of badgeList) {
      if (badge.earned && !previousEarned.has(badge.id)) {
        void notifyBadgeEarned('Huy hieu moi', 'Ban vua mo khoa "${badge.title
        }"');
      }
    }
  }
  return badgeList;
});
}, [userId]);

```

Giải thích từng phần:

1. `Promise.all` — Tải dashboard, badge, prefs **cùng lúc** (3 request song song).
2. `setSummary(dashboard)` - Cập nhật greeting, lưới chỉ số nhanh và biểu đồ. `StatsGrid` tự tải dữ liệu sức khỏe riêng; `WorkoutCard` hiện là nội dung tĩnh.
3. `setBadges` **với callback** — So sánh badge **cũ** vs **mới**:
 - Tạo Set các id badge đã đạt trước đó (`previousEarned`).
 - Duyệt `badgeList` mới: nếu badge đạt (`earned`) mà **chưa có trong Set** -> gọi `notifyBadgeEarned`.
 - Chỉ bắn thông báo khi `badge_notifications_enabled === true`.
4. `useCallback(..., [userId])` — Hàm chỉ tạo lại khi `userId` đổi, tránh re-render vô ích.
5. `refreshKey` (prop từ `App.tsx`) — Tăng sau khi hoàn thành buổi tập -> `useEffect` gọi lại `loadDashboard`.

Sai lệch cần lưu ý: ở lần load đầu, `previous` là `[]`, nên mọi badge đã đạt đều vắng trong `previousEarned`. Vì thế code có thể thông báo lại badge cũ; phép so sánh hiện tại chưa chứng minh badge vừa được insert trong lần sync này.

0.10.5 WeeklyProgressChart — biểu đồ 7 ngày

Input: `days: WeeklyWorkoutDay[]` từ `summary.weekly_workouts`.

Logic chính:

```

const chartDays = useMemo(
  () => (days.length === 7 ? days : buildFallbackDays()),
  [days],
);

```



```
const totalWorkouts = chartDays.reduce((sum, day) => sum + day.workouts, 0);
const hasActivity = totalWorkouts > 0;
const maxWorkouts = Math.max(...chartDays.map(d => d.workouts), 1);
```

Biến	Ý nghĩa
chartDays	Luôn 7 ngày (fallback nếu input sai)
hasActivity	false -> hiển thị empty state “Chưa có buổi tập”
maxWorkouts	Ngày tập nhiều nhất = 100% chiều cao cột; tối thiểu 1 để tránh chia cho 0

Tính chiều cao cột:

```
const fillHeight = active
  ? Math.max(12, (day.workouts / maxWorkouts) * CHART_HEIGHT)
  : 0;
```

- Ngày có tập (active): cột cao tỉ lệ workouts / maxWorkouts, tối thiểu 12px.
- Ngày không tập: hiển thị chấm nhỏ (barEmpty) thay vì cột mờ 8px (bug cũ đã fix).
- isToday: viền xanh lá (barTrackToday) + nhãn ngày đậm.

0.10.6 isBadgeEarned(badge, stats) — badgeCalculations.ts

Mục đích: Hàm **thuần** (không gọi mạng) — kiểm tra 1 huy hiệu đã đủ điều kiện chưa. Để unit test.

Code:

```
export function isBadgeEarned(badge: Badge, stats: BadgeStats): boolean {
  switch (badge.criteria_type) {
    case 'onboarding': return stats.onboardingComplete;
    case 'workout_sessions': return stats.workoutSessions >= badge.criteria_value;
    case 'streak': return stats.currentStreak >= badge.criteria_value;
    case 'water_goal_days': return stats.waterGoalDays >= badge.criteria_value;
    case 'nutrition_goal_days': return stats.nutritionGoalDays >= badge.criteria_value;
    case 'calories_burned_day': return stats.maxCaloriesBurnedDay >= badge.criteria_value;
```

```

    default:                return false;
  }
}

```

Ví dụ: Badge “Kiên trì 3 ngày” có `criteria_type: 'streak'`, `criteria_value: 3`. - stats.
`.currentStreak = 2` -> false (chưa đạt). - stats.`currentStreak = 3` -> true (đạt).

0.10.7 loadBadgeStats(userId) — badgeService.ts

Mục đích: Gom một lần tất cả thông kê cần để kiểm tra huy hiệu.

Các chỉ số thu thập:

Chỉ số	Cách tính
onboardingComplete	profiles.onboarding_completed === true
workoutSessions	COUNT(*) từ workout_sessions
currentStreak	user_streaks.current_streak
waterGoalDays	Đếm ngày water_ml >= water_goal_ml (+ hôm nay nếu đạt)
nutritionGoalDays	Đếm ngày calories_consumed >= daily_calorie_goal
maxCaloriesBurnedDay	Gom calo theo workout_date, lấy max

Lưu ý: Hôm nay được tính riêng qua `getDailyNutrition` / `getTodayWater` vì có thể chưa có dòng trong bảng daily (chưa ghi trong ngày).

`nutritionGoalDays` so sánh mọi ngày lịch sử với **mục tiêu calo hiện tại** của profile, không lưu snapshot mục tiêu theo từng ngày. Nếu user đổi mục tiêu, số ngày đạt chuẩn trong quá khứ có thể thay đổi.

0.10.8 syncUserBadges(userId) — cập huy hiệu tự động

Code:

```

// Rut gon tu badgeService.ts; code that co kiem tra badgesError.
export async function syncUserBadges(userId: string): Promise<BadgeWithStatus[]> {
  const [{ data: badges, error: badgesError }, stats, { data: earned }] = await
    Promise.all([

```

```

    supabase.from('badges').select('*').order('sort_order', { ascending: true }),
    loadBadgeStats(userId),
    supabase.from('user_badges').select('badge_id').eq('user_id', userId),
  ]);

  if (badgesError || !badges?.length) return getBadgesWithStatus(userId);

  const earnedIds = new Set((earned ?? []).map(r => r.badge_id));
  const toAward = badges.filter(b => !earnedIds.has(b.id) && isBadgeEarned(b,
    stats));

  if (toAward.length > 0) {
    const { error } = await supabase.from('user_badges').insert(
      toAward.map(b => ({ user_id: userId, badge_id: b.id })),
    );
    if (error && error.code !== '23505') console.warn(error.message);
  }
  return getBadgesWithStatus(userId);
}

```

Luồng:

1. Tại song song: danh mục badge + stats user + badge đã đạt
2. earnedIds = Set các badge_id đã có trong user_badges
3. toAward = badge chưa có VÀ isBadgeEarned() = true
4. INSERT toAward vào user_badges
5. Tra getBadgesWithStatus() - danh sách đầy đủ kèm earned/earned_at

Tại sao bỏ qua lỗi 23505? Hai lần gọi `syncUserBadges` đồng thời có thể cùng cố insert một badge. Tuy nhiên đây là insert theo batch; nếu batch gặp conflict, cần kiểm thử để chắc các badge khác trong batch không bị bỏ lỡ. Cách rõ ràng hơn là dùng `upsert/ON CONFLICT DO NOTHING` phù hợp với policy.

0.10.9 getNotifications() — lazy-load an toàn Expo Go

Mục đích: Chỉ load expo-notifications khi **không** chạy trên Expo Go.

```

const isExpoGo = Constants.executionEnvironment === ExecutionEnvironment.
  StoreClient;

async function getNotifications(): Promise<NotificationsModule | null> {
  if (isExpoGo) return null; // Expo Go -> không dùng notification

```

```

if (notificationsModule === undefined) {
  try {
    notificationsModule = await import('expo-notifications'); // import xong,
    lan dau
    notificationsModule.setNotificationHandler({ /* hien alert + am thanh */ });
  } catch {
    notificationsModule = null;
  }
}
return notificationsModule;
}

```

Pattern lazy-load: - undefined = chưa thử load. - null = đã thử import nhưng lỗi. - NotificationsModule = đã load thành công và được cache. - Trên Expo Go, hàm trả null sớm; biến cache vẫn giữ nguyên.

-> Mọi hàm notification gọi getNotifications() trước; nếu null -> return sớm, **không crash**.

0.10.10 updateNotificationPreferences(userId, prefs) — bật/tắt nhắc

Mục đích: Cập nhật cài đặt nhắc nhở — vừa lập lịch trên máy, vừa ghi cở lên server.

Luồng xử lý:

1. Doc prefs hien tai: getNotificationPreferences(userId)
2. Merge: next = { ...current, ...prefs }
3. Neu bat nhac nuoc (true):
 - > enableWaterReminders() - xin quyen + lap 8 thong bao lap ngay
 Neu tat (false):
 - > disableWaterReminders() - huy tat ca id da luu trong AsyncStorage
4. Neu bat nhac tap (true):
 - > requestPermissions() - xin quyen OS
 - > (Android) tao notification channel
 - > scheduleWorkoutReminder(wakeup_time) - lap lich CALENDAR lap ngay
 Neu tat -> cancelWorkoutReminder()
 - Neu doi gio khi dang bat -> scheduleWorkoutReminder(gio moi)
5. Ghi 3 co boolean va 'wakeup_time' len 'profiles' (Supabase)
6. Tra next prefs

Luồng này không phải transaction giữa OS và database: lịch local được đổi trước, rồi

mới update profiles. Nếu update Supabase thất bại, lịch trên máy và cở server có thể lệch nhau.

scheduleWorkoutReminder(wakeupTime) chi tiết:

```
await cancelWorkoutReminder(); // huy lịch cũ (tranh trùng)
const { hour, minute } = parseWakeupTime(wakeupTime); // "06:30" -> {6, 30}
const id = await Notifications.scheduleNotificationAsync({
  content: { title: 'ỜGi ập ệluyn ', body: '...', sound: true },
  trigger: { type: CALENDAR, hour, minute, repeats: true }, // lặp mỗi ngày
});
await AsyncStorage.setItem(WORKOUT_REMINDER_ID_KEY, id); // lưu id để hủy sau
```

0.10.11 syncAllRemindersOnLaunch(userId) — khôi phục khi mở app

Gọi từ: App.tsx sau khi xác nhận user đã onboarding.

```
export async function syncAllRemindersOnLaunch(userId: string): Promise<void> {
  if (isExpoGo) return;

  await syncWaterRemindersOnLaunch(userId); // doc có water -> dat lại 8 cốc nếu
    bat

  const prefs = await getNotificationPreferences(userId);
  if (prefs.workout_reminder_enabled) {
    const granted = await requestPermissions();
    if (granted) await scheduleWorkoutReminder(prefs.wakeup_time);
  }
}
```

Tại sao cần? Khi id trong AsyncStorage không còn, app có thể dựa vào cở server để thử đặt lại lịch. Kết quả vẫn phụ thuộc quyền OS và kết nối mạng; lỗi trong App.tsx hiện chỉ được ghi bằng console.error.

0.10.12 RPC get_dashboard_summary() — SQL trên server

Phần quan trọng nhất — snapshot 7 ngày:

```
from generate_series(v_today - 6, v_today, interval '1 day') as day
left join lateral (
  select count(*)::int as count, coalesce(sum(total_calories),0) as calories
```

```

from public.workout_sessions
where user_id = v_user_id and workout_date = day::date
) ws on true

```

Giải thích: - generate_series tạo 7 ngày liên tiếp (hôm nay - 6 -> hôm nay). - LEFT JOIN LATERAL với subquery đếm session theo từng ngày. - Ngày **không có buổi tập** vẫn xuất hiện với workouts = 0, calories_burned = 0. - Việc giữ đủ ngày đến từ generate_series kết hợp subquery aggregate luôn trả một dòng; không nên khẳng định mọi INNER JOIN đều làm mất ngày nếu chưa xét hình dạng subquery. - v_today := current_date theo timezone database. Cần cấu hình/chuẩn hóa timezone để khớp với ngày local trên thiết bị.

Bảo mật: - v_user_id := (select auth.uid()) — chỉ lấy data của user đang đăng nhập. - SECURITY INVOKER — chạy với quyền user, RLS vẫn áp dụng. - if v_user_id is null then raise exception — từ chối nếu chưa đăng nhập.

0.10.13 Tóm tắt hàm

- getDashboardSummary (dashboardService): nhận userId, trả DashboardSummary; gọi khi mở hoặc refresh Dashboard.
- normalizeWeeklyWorkouts (dashboardService): nhận raw: unknown, trả mảng 7 ngày; chạy sau RPC.
- loadDashboard (DashboardScreen): tải song song summary, badge và prefs rồi cập nhật state.
- isBadgeEarned (badgeCalculations): nhận badge và stats, trả boolean cho từng điều kiện.
- syncUserBadges (badgeService): kiểm tra/cập badge khi Dashboard load.
- getNotificationPreferences và updateNotificationPreferences (notificationService): đọc/ghi cài đặt modal.
- syncAllRemindersOnLaunch (notificationService): thử khôi phục lịch sau login và onboarding.
- notifyBadgeEarned (notificationService): phát local notification tức thời; hiện chưa tự xin quyền.
- get_dashboard_summary (SQL RPC): lấy user từ JWT và trả jsonb.

Tài liệu giải thích module Dashboard, Notification & Badges — FitLife. Phiên bản để hiểu + giải thích code cho nghiệm thu và báo cáo nhóm.